

FIT TRACK

AI-Based Fitness Tracking and Personalized Fitness Recommendations

Submitted by

Team ID: SWTID-2026-1838

Team Leader: SUJATHA N

Team Member: GOMATHI V

Team Member: MAYAVATHI V

Team Member: GAYATHRI D

Department of Computer Science

Chellammal Women's College

2026 - 2027

1.INTRODUCTION

FitTrack AI is a secure and scalable RESTful backend application designed for fitness tracking and personalized fitness guidance. The system uses Node.js, Express.js, MongoDB, and Mongoose to provide APIs for authentication and workout management.

Authenticated users can create, view, retrieve, update, delete, and search workout records. The application also provides AI-assisted fitness guidance using Google Gemini.

The project follows a modular MVC-based architecture. Authentication is implemented using JSON Web Tokens (JWT), while passwords are protected using bcrypt.js. Protected APIs validate the JWT before allowing access.

The AI service uses user-provided age, fitness goal, and workout experience to generate personalized recommendations. It can also analyze workout statistics such as total workouts, average workout duration, and calories burned to provide fitness insights.

2. PROBLEM STATEMENT

Fitness information is often recorded manually, making it difficult to maintain an organized workout history and understand progress over time.

The project addresses the need for a centralized backend where users can securely maintain workout information. It also addresses the need for efficient workout searching, personalized fitness recommendations, and performance insights.

The system provides secure authentication, centralized workout records, CRUD operations, search functionality, and AI-assisted guidance. This reduces the need for users to manage separate records and manually interpret their workout history.

FitTrack AI therefore combines fitness tracking with AI assistance in a single RESTful backend application.

3. OBJECTIVES AND PROPOSED SOLUTION

3.1 Objectives

- Provide secure user registration and login.
- Protect user-specific APIs.
- Manage workout records.
- Search workouts efficiently.
- Provide AI-based recommendations and fitness insights.

3.2 Proposed Solution

The proposed solution uses a centralized backend connected to MongoDB. Users authenticate using JWT and then access protected workout APIs.

Workout records can be added, viewed, retrieved by ID, updated, deleted, and searched. The AI service communicates with Google Gemini to generate personalized recommendations and analyze workout statistics.

The backend also uses validation, centralized error handling, and modular services so that authentication, workout management, database access, and AI processing remain organized.

4. SOFTWARE AND HARDWARE REQUIREMENTS

4.1 Software Requirements

Component	Requirement
Operating System	Windows / macOS / Linux
Node.js	v16 or above
npm	v8 or above
Backend Framework	Express.js
Database	MongoDB
ODM	Mongoose
API Testing	Postman
Development Environment	Visual Studio Code
AI Service	Google Gemini API

Project dependencies include express, mongoose, dotenv, bcryptjs, jsonwebtoken, and cors. Nodemon can be used during development.

Environment variables are used for configuration and secrets, including PORT, MONGO_URI, JWT_SECRET, and GEMINI_API_KEY.

4.2 Hardware Requirements

- Processor: Intel i5 / AMD Ryzen 5 or equivalent
- RAM: 8 GB or above
- Storage: Approximately 1 GB available for the project environment

5. SYSTEM ARCHITECTURE

FitTrack AI follows a layered MVC architecture. A client such as a React application, mobile application, or Postman sends HTTP requests to the Express server.

The request passes through API routes and, where required, authentication middleware. Controllers handle request validation and coordinate the required business operation. Services provide reusable application logic such as JWT handling, password operations, and Gemini AI communication. Mongoose connects the application to MongoDB for persistent storage.

5.1 Request Flow

Client → Express Server → API Routes → Auth Middleware → Controllers → Services → MongoDB / Gemini AI → JSON Response

The architecture separates responsibilities so that authentication, workout processing, database operations, and AI processing are handled by appropriate modules.

Client	Express Server	API Routes	Auth Middleware
Controllers	Services	MongoDB	Gemini AI

6. AUTHENTICATION AND SECURITY

User authentication is implemented using JWT. During registration, the user provides the required account information, and the password is protected using bcrypt.js.

During login, the server verifies the credentials and issues a JWT. Protected requests include the token in the Authorization header.

The authentication middleware verifies the token signature, decodes the user information, and attaches the authenticated user information to the request. Unauthorized requests are rejected.

Security-related implementation also includes environment variables for secrets, input validation, CORS configuration, and centralized error handling. Access to workout data is tied to the authenticated user.

The documented project uses one authenticated user role with permissions for profile access, workout management, search, AI recommendations, and fitness insights.

7. API ROUTES AND WORKOUT MANAGEMENT

The API uses REST principles and standardized JSON responses.

Area	Documented Operations
Authentication	User registration, login, and profile access
Workout Management	Add workout, get all workouts, get a workout by ID, update workout, delete workout
Workout Search	Search using workout name, category, or date

AI Features	Personalized recommendations and fitness insights
-------------	---

Each workout belongs to the authenticated user. The workout controller receives the request, validates the data, performs the required operation through the service/database layer, and returns the appropriate JSON response.

The documented project focuses on authentication, workout CRUD/search, and AI endpoints rather than separate step, water, calorie, and goal collections.

8. DATABASE DESIGN

MongoDB is used as the primary database, and Mongoose is used for schema definition and database interaction.

The project documentation identifies two primary collections: Users and Workouts. The User-to-Workout relationship is one-to-many: one user can have many workout records, while each workout belongs to one authenticated user.

8.1 User Collection

Field	Description / Rule
_id	User identifier
name	Mandatory
email	Unique
password	Stored using crypt protection
created At	Mongoose timestamp
updated At	Mongoose timestamp

8.2 Workout Collection

Field	Description / Rule
_id	Workout identifier
user reference	Reference to the authenticated user
workout Name	Required
category	Required
duration	Must be greater than zero
calories Burned	Cannot be negative

workout Date	Required
created At	Mongoose timestamp

Mongoose timestamps and validation support consistent storage and retrieval of records.

9. MVC AND BACKEND MODULES

The project follows the Model-View-Controller pattern.

9.1 Model

Mongoose models represent the User and Workout database structures and their validation rules.

9.2 View

Because this is a REST API backend, the view is represented by API consumers such as a React client, mobile application, or Postman. The backend returns JSON rather than rendering traditional web pages.

9.3 Controller

Controllers handle incoming requests, validation, business coordination, database operations, and AI requests, and return responses to the client.

9.4 Backend Modules

The project is organized into modules for server configuration, database configuration, authentication, workout management, AI recommendation, middleware, and error handling. This modular structure supports maintainability and separation of responsibilities.

9.5 Benefits of MVC and Modular Design.

- **Separation of Responsibilities**

Services and middleware can be reused by multiple API endpoints when required.

- **Maintainability**

The modular structure provides a foundation for adding new fitness features in future.

- **Testing and Debugging**

Changes in one module can be made with less impact on other parts of the application.

- **Reusability**

Separate controllers, services, and models make individual components easier to test and debug.

- **Scalability**

The modular structure provides a foundation for adding new fitness features in future.

10. GEMINI AI - PERSONALIZED RECOMMENDATIONS

Google Gemini is integrated through a dedicated AI service.

The recommendation feature accepts three documented user inputs: Age, Fitness Goal, and Experience Level.

The AI service sends the relevant information to Gemini and returns personalized fitness guidance. The documented output can include a personalized plan, weekly suggestions, recommendations suitable for beginner, intermediate, or advanced experience, motivational guidance, and safety tips.

The AI functionality is an important part of FitTrack AI because it extends the project beyond simple workout record management.

The AI service is separated from the main workout controller so that AI communication can be managed as a dedicated service module.

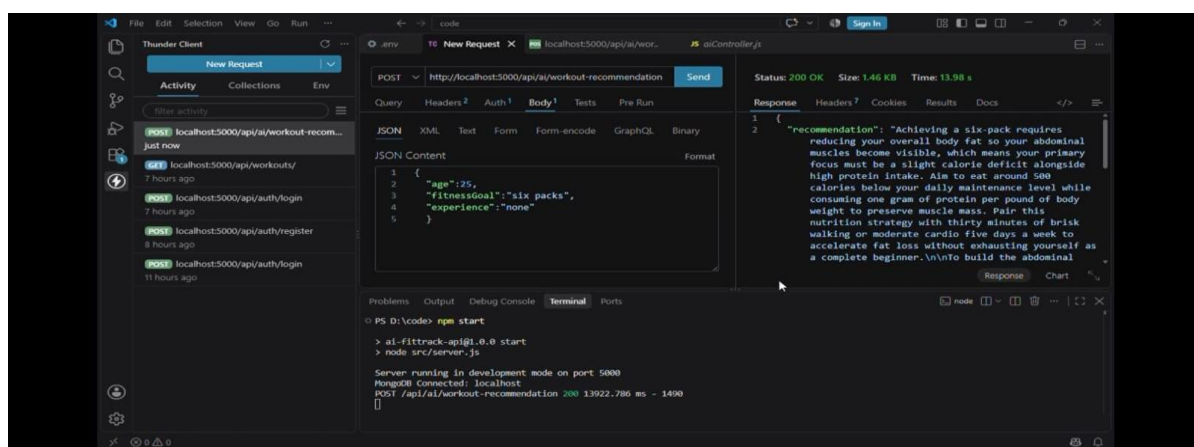


Figure: Gemini AI Recommendation / Output

11. AI FITNESS INSIGHTS

FitTrack AI also provides AI-assisted fitness insights based on workout statistics.

The documented statistics used for analysis are total workouts, average workout duration, and total calories burned.

These values are provided to the AI service for interpretation. The documented output includes performance analysis, improvement suggestions, motivational advice, and a progress summary.

The feature helps convert stored workout history into understandable fitness information instead of requiring the user to manually calculate or interpret all statistics.

Both AI recommendations and AI fitness insights are returned through REST API endpoints as JSON responses.

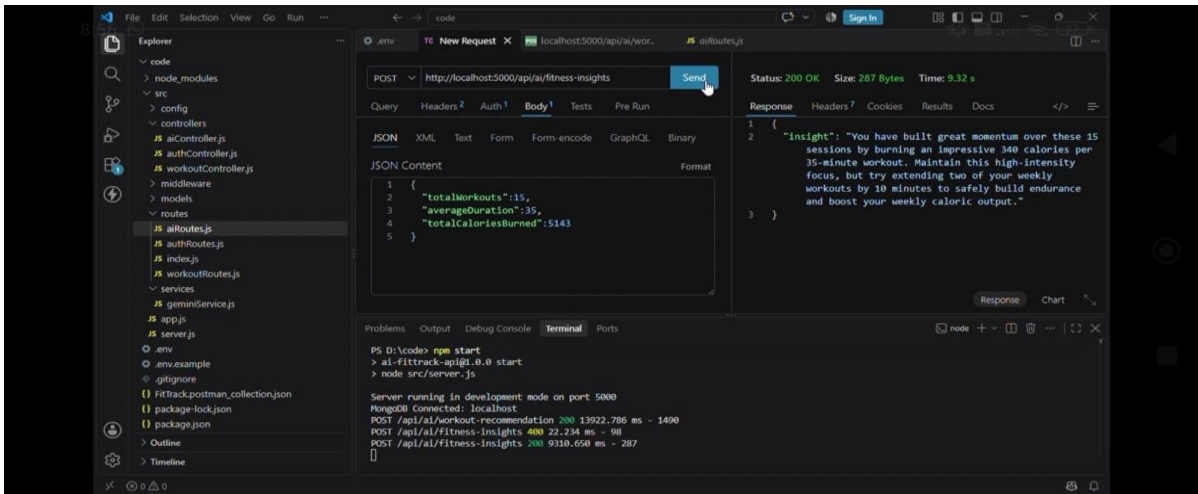


Figure: AI Fitness Insights / Output

10

12. PROJECT SETUP AND TESTING

The project setup uses Node.js and npm. The server project is initialized with npm, and the required dependencies are installed. Environment variables are configured for the server port, MongoDB connection, JWT secret, and Gemini API key.

MongoDB connection is configured through Mongoose. User and Workout schemas are defined with the documented validation rules.

12.1 Testing Tools

API testing is performed using Postman or Thunder Client.

12.2 Testing Order

- Authentication
- Workout operations
- AI features

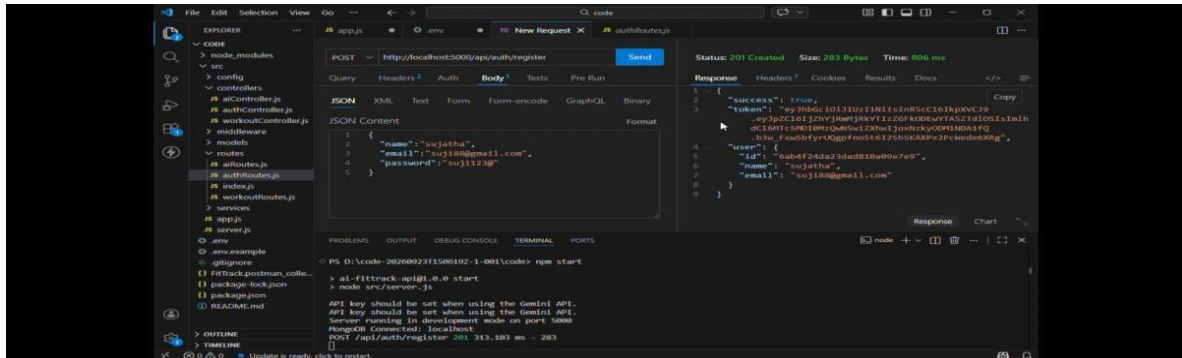
12.3 Testing Coverage

Test Area	Coverage
Authentication	Registration, login, and profile access
Workout Operations	Adding, retrieving, searching, updating, and deleting workout records
AI Features	Workout recommendation and fitness insight endpoints

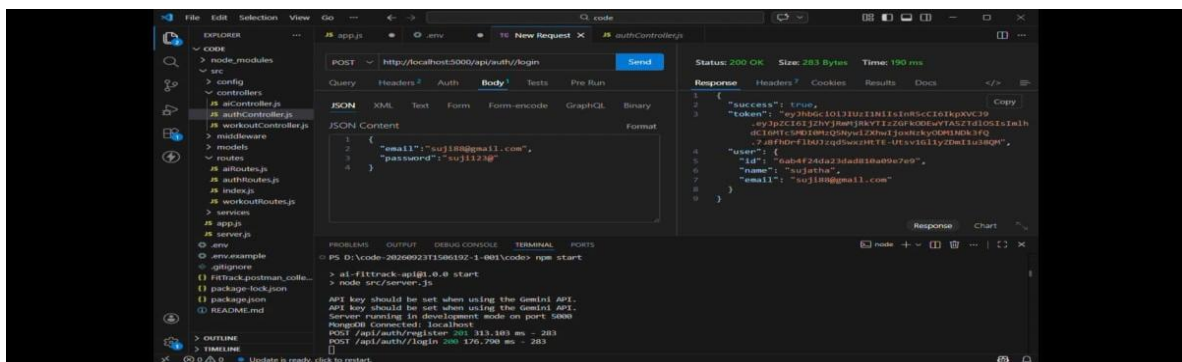
Testing confirms that protected routes require valid authentication and that the main API operations return the expected JSON responses.

13. API TESTING / IMPLEMENTATION OUTPUT

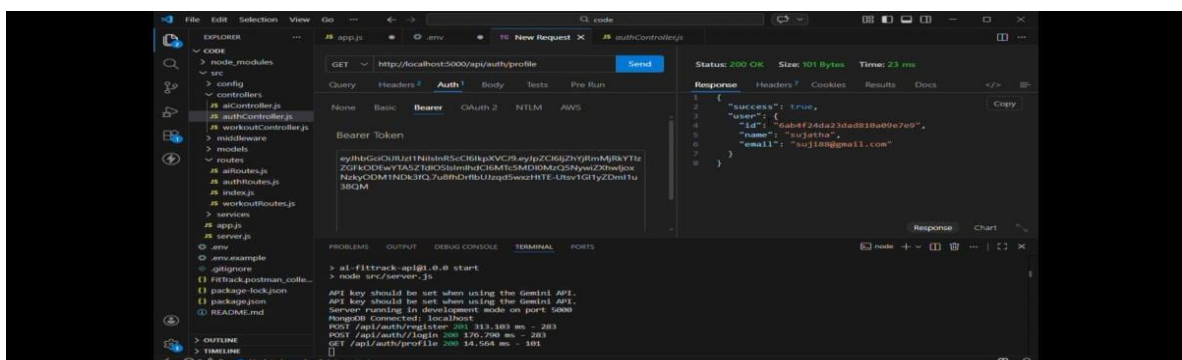
The implementation was tested through API requests using the documented authentication and AI endpoints. The screenshots below represent the testing flow and JSON responses.



Registration API - successful user creation



Login API – successful authentication and token response



Profile API – authenticated user profile response

14.DEMO VIDEO LINK

https://1drv.ms/v/c/1BBA9C40FFA1D19B/IOBbLUdfNZSMTKI6XBaG8C7FAS_qdhErglONvsfGHxsxSY0

15. KEY FEATURE

- **Secure User Authentication**
JWT-based secure login and registration
- **Workout Management**
Add, view, update and delete workout records
- **Workout Search**
Search workouts by name, category and date
- **AI-Powered Personalized Recommendations**
Gemini AI gives recommendations based on age, goal and experience
- **AI Fitness Insights**
Analyses workout statistics and provides progress insights
- **Centralized Workout Data**
MongoDB stores user and workout information securely

16. FUTURE SCOPE

The project documentation identifies possible future additions such as nutrition support, BMI-related features, wearable integration, reminders, and analytics dashboards.

These additions are listed as possible future enhancements to extend the documented fitness tracking and AI-assisted backend capabilities.

- **Nutrition Support**
Add nutrition and food tracking to provide a more complete fitness management system.
- **BMI-Related Features**
Include BMI calculation and related information as an additional fitness reference.
- **Wearable Integration**
Connect fitness watches and wearable devices to collect activity and workout data.
- **Reminders and Notifications**
Provide reminders for scheduled workouts and other fitness activities.
- **Analytics Dashboard**
Display workout statistics and progress through charts and an easy-to-understand dashboard.
- **Advanced Fitness Analytics**
Analyse workout history to identify performance trends and provide detailed summaries.
- **Improved AI Assistance**
Extend AI capabilities to generate more personalized guidance using stored workout history.
- **Mobile Application**
Develop a mobile application so users can access their fitness information conveniently.
- **Progress Tracking**
Add visual progress tracking using charts, statistics, and historical workout data.
- **Goal Management**
Allow users to set, monitor, and update personal fitness goals.

These enhancements can extend FitTrack AI from a workout-record backend into a broader fitness tracking and AI-assisted health management platform.

17. CONCLUSION

FitTrack AI combines secure authentication, workout management, database storage, and AI-powered fitness assistance in a modular RESTful backend. The system allows authenticated users to manage workout history and receive personalized recommendations and insights through Google Gemini.

The documented implementation uses Node.js, Express.js, MongoDB, Mongoose, JWT, bcrypt.js, and Gemini AI, with modular controllers, services, middleware, and database models.

The project therefore provides a centralized backend structure for authentication, workout CRUD and search operations, AI recommendations, and fitness insights.